Data Engineering Career Roadmap

From BI Support to Fabric Data Engineer — Day-by-Day Plan

Starting point: SSRS/PBIRS/SSAS/AAS support engineer, Microsoft Fabric Data Engineer Associate (DP-700) certified **Target:** Job-ready Fabric/Azure Data Engineer with a live GitHub portfolio **Pace:** 5–9 hrs/week | **Total duration:** ~4 months (16 weeks)

How to use this plan

- Each project gets its own GitHub repo. Push code and commit notes **as you go**, not at the end — your commit history is part of your proof of work.
- Every project ends with a README containing: problem statement, architecture diagram, setup steps, screenshots/GIF, and "what I'd improve with more time."
- Don't perfect Project 1 and 2 — ship them fast. Perfection matters more from Project 3 onward.
- Proficiency checkpoints are included so you can honestly self-assess before moving on.

Phase 0: Environment Setup (Days 1–3)

Day	Task
1	Set up a Microsoft Fabric trial workspace. Set up GitHub account structure (one repo per project, consistent naming: `fabric-project-01-ingestion`, etc.)
2	Install/configure: VS Code, Git, Python 3.x, PySpark locally (optional but useful for testing outside Fabric notebooks)
3	Review Fabric Lakehouse vs Warehouse vs Data Factory conceptually — write a one-page personal notes doc (this becomes interview prep material later)

Proficiency checkpoint: You can explain the difference between a Lakehouse and a Warehouse in Fabric without looking it up.

Project 1: Basic Ingestion Pipeline (Week 1)

Goal: Move data into Fabric using Data Factory into a Lakehouse bronze table.

Day	Task
1	Pick a dataset (e.g., a public retail sales CSV or weather API). Explore its structure.
2	Create a Fabric Lakehouse. Manually upload the file once to understand the bronze folder structure.
3	Build a Data Factory pipeline: Copy Data activity from source (HTTP/CSV) into the Lakehouse bronze table.
4	Add a schedule trigger (daily/hourly). Test a manual run and a triggered run.
5	Add basic pipeline monitoring — check run history, simulate a failure and observe error handling.
6	Write the README: architecture diagram, pipeline screenshot, what "bronze" means in this context.
7	Push to GitHub. Buffer day for cleanup.

Proficiency checkpoint: You can build a scheduled ingestion pipeline from scratch in under 30 minutes.

Project 2: PySpark Transformation Notebook (Weeks 2–3)

Goal: Prove you can code transformations, not just configure pipelines.

Day	Task
8	Open a Fabric notebook attached to your Lakehouse. Load the bronze table as a Spark DataFrame.
9	Practice core PySpark operations: `select`, `filter`, `withColumn`, `groupBy`, `agg`.
10	Handle data quality issues: nulls, duplicates, type mismatches. Write reusable functions for each.
11	Enforce a schema explicitly (`StructType`) instead of relying on inference.
12	Write the cleaned output to a **silver** Delta table. Verify with `DESCRIBE HISTORY` (Delta Lake versioning).
13	Add basic logging/print statements documenting row counts before/after each transformation step.
14	Write README explaining each transformation decision (this is what interviewers probe — be ready to justify each choice). Push to GitHub.

Proficiency checkpoint: You can explain *why* you chose each transformation, not just what it does.

Project 3: Full Medallion + Power BI Report (Weeks 4–6)

Goal: Your first complete, demo-able project — this is your headline repo.

Week	Focus
4	Build the **gold layer**: aggregate silver data into business-ready tables (e.g., daily/monthly summaries, KPIs). Use `groupBy`/window functions for rolling metrics.
5	Connect Power BI directly to the gold Lakehouse tables. Build 1 clean report page: KPI cards, a trend chart, a breakdown visual. Apply your existing BI/DAX strength here.
6	Polish: add a bookmark or slicer, record a short screen-capture GIF of the report, write full README with end-to-end architecture diagram. Push to GitHub and **pin this repo** on your profile.

Proficiency checkpoint: You can walk someone through the full bronze→silver→gold→report flow in under 5 minutes, live.

Project 4: Incremental Warehouse + CI/CD (Weeks 7–10)

Goal: Show engineering maturity most self-taught candidates skip.

Week	Focus
7	Design a proper **fact/dimension model** in Fabric Warehouse for your dataset (star schema).
8	Implement **incremental loading**: use a watermark column (e.g., last modified date) so re-runs only process new/changed rows, not a full refresh.
9	Set up a **GitHub Actions workflow**: on push, validate notebook/pipeline JSON, and (where feasible) trigger a deployment step. Learn Fabric's deployment pipelines/Git integration alongside this.
10	Add basic alerting concepts (e.g., a simple failure notification pattern). Write README covering the incremental logic and CI/CD flow with a diagram. Push to GitHub.

Proficiency checkpoint: You can explain incremental loading and CI/CD for data pipelines to a non-technical interviewer in plain language.

Project 5: Capstone — Legacy BI to Fabric Migration (Weeks 11–16)

Goal: Your differentiator — nobody else applying alongside you can credibly build this.

Week	Focus
11	Set up a small legacy-style source: a SQL Server database + a basic SSAS tabular model (or simulate with a well-structured SQL schema if SSAS isn't available to you).
12	Build a basic report against it (simulate an "old world" SSRS-style report, even a simple one).
13	Migrate the pipeline: ingest the same source into Fabric, rebuild the medallion architecture.
14	Recreate the semantic model as a Fabric semantic model / Power BI dataset. Rebuild the report in Power BI, connected live to Fabric.
15	Enable Copilot in Fabric (or Power BI Copilot) on top of your gold layer. Test natural-language queries against your own data.
16	Write the capstone documentation: migration decisions, what changed, performance differences, a "lessons learned / migration checklist." This becomes your strongest interview narrative. Push, pin, and cross-link all 5 repos from a top-level GitHub profile README.

Proficiency checkpoint: You can narrate this project for 15–20 minutes in an interview without running out of substantive detail.

Ongoing: Future-Facing Concepts to Layer In

Work these into your study time throughout — don't wait until the end. They're what separate "did a Fabric course" from "understands where the field is going."

Concept	Why it matters now
Fabric AI Skills / Copilot integration	Companies increasingly want engineers who build data foundations *for* AI, not just BI. You're doing this in Project 5 — go deeper.
Real-Time Intelligence (Eventstream, KQL Database)	Fabric's newest capability area; fewer candidates have hands-on experience — strong differentiator.
RAG pipeline basics (vector embeddings, vector search)	Even conceptual fluency here makes you far more hireable for "AI-adjacent" data roles.
Data governance (Purview integration, OneLake security, row-level security)	Enterprises care deeply about this; your regulated-industry (banking/healthcare-adjacent) background is a genuine asset here.
Cost/performance optimization (Spark tuning, partitioning, caching)	Interviewers love asking "how would you optimize this" — have real examples from your own projects.
DataOps mindset (testing pipelines, data quality frameworks, observability)	Signals engineering maturity beyond tutorial-level work.

Milestone Summary

Milestone	Target Week	Signals
First repo live	Week 1	Momentum, basic Fabric fluency
Can code real transformations	Week 3	Not just a "config-only" candidate
One polished, demo-able project	Week 6	Ready to start applying/interviewing
CI/CD + incremental logic understood	Week 10	Above-average portfolio depth
Capstone complete	Week 16	Differentiated, interview-ready story

Suggestion: Start applying to roles around Week 6–7, once Project 3 is live. Don't wait for all five — a strong 3-project portfolio with active weekly commits shows momentum, which matters as much as completeness.